Exercícios sobre Coleções em Clojure – Com Gabarito

Exercício 1 – Primeiro e último nome (vetor)

Crie um vetor de nomes e exiba o primeiro e o último elemento.

```
(ns ex_col1.core)

(def nomes ["Ana" "Bruno" "Carlos" "Diana" "Eduardo"])

(defn -main []
  (println "Primeiro:" (first nomes))
  (println "Último:"   (peek nomes)))
```

Exercício 2 – Adicionar nome com conj (vetor)

Adicione um novo nome ao vetor original com conj, exibindo o vetor antigo e o novo.

```
(ns ex_col2.core)

(def nomes ["Ana" "Bruno" "Carlos"])

(defn -main []
  (let [novos (conj nomes "Fernanda")]
    (println "Original:" nomes)
    (println "Com novo elemento:" novos)))
```

Exercício 3 – Mapa de aluno com status

Crie um mapa representando um aluno e adicione a chave :status baseada na :nota (>= 60 aprovado, caso contrário reprovado).

```
(ns ex_col3.core)

(def aluno {:nome "João" :nota 72 :curso "Engenharia"})

(defn -main []
  (let [status (if (>= (:nota aluno) 60) :aprovado :reprovado)
        aluno2 (assoc aluno :status status)]
    (println aluno2)))
```

Exercício 4 – Soma de um vetor com reduce

Dado um vetor de números, calcule a soma utilizando reduce.

```
(ns ex_col4.core)

(def numeros [10 20 30 40])

(defn -main []
```

```
  (println "Soma:" (reduce + numeros)))
```

Exercício 5 – Conjunto sem repetição

Demonstre que um conjunto não aceita repetição ao tentar inserir um elemento que já existe.

```
(ns ex_col5.core)

(def conjunto-inicial #{1 2 3})

(defn -main []
  (let [novo (conj conjunto-inicial 2)]
    (println "Original:" conjunto-inicial)
    (println "Após conj 2:" novo)))
```

Exercício 6 – Filtrar pares (vetor)

Escreva uma função que receba um vetor e retorne um novo vetor apenas com os valores pares.

```
(ns ex_col6.core)

(defn apenas-pares [v]
  (into [] (filter even? v)))

(defn -main []
  (println (apenas-pares [1 2 3 4 5 6 7 8])))
```

Mini Desafio – Estatísticas de idades (vetor)

Armazene as idades de 5 pessoas em um vetor e calcule: média, mais novo, mais velho e quantos têm idade ≥ 18.

```
(ns ex_col_desafio.core)

(def idades [18 22 15 40 30])

(defn media [v]
  (double (/ (reduce + v) (count v))))

(defn -main []
  (println "Idades:" idades)
  (println "Média:" (media idades))
  (println "Mais novo:" (apply min idades))
  (println "Mais velho:" (apply max idades))
  (println "Maiores ou iguais a 18:" (count (filter #(>= % 18) idades))))
```